

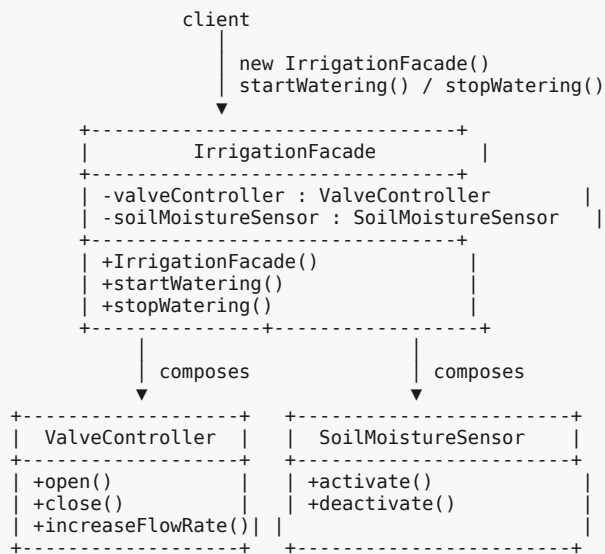
Facade Design Pattern

Intent

Provide a unified interface to a set of interfaces in a subsystem. Facade defines a higher-level interface that makes the subsystem easier to use.

This is the **canonical GoF Facade**: an `IrrigationFacade` that owns and coordinates two independent subsystem classes (`ValveController`, `SoilMoistureSensor`), publishing two intention-revealing operations (`startWatering()`, `stopWatering()`). The client never constructs, imports, or calls a subsystem class — it only ever talks to `new IrrigationFacade()`.

UML class diagram



The players

<code>facade/SmartIrrigationDesignPattern</code>	the client – talks only to <code>IrrigationFacade</code>
<code>facade/IrrigationFacade</code>	the Facade – owns and orchestrates the subsystem
<code>facade/subsystem/ValveController</code>	subsystem class – fine-grained valve controls
<code>facade/subsystem/SoilMoistureSensor</code>	subsystem class – fine-grained sensor controls

`ValveController` and `SoilMoistureSensor` live under the `subsystem` subpackage because, from the Facade's point of view, they are "the subsystem" — the pieces a client would otherwise have to know how to construct and sequence correctly.

Code walkthrough

`ValveController` — subsystem class

```
package com.design.patterns.facade.subsystem;

public class ValveController {

    public void open() {
        System.out.println("-----Valve Controller Open");
    }

    public void close() {
        System.out.println("-----Valve Controller Closed");
    }

}
```

```

    public void increaseFlowRate() {
        System.out.println("-----Valve Controller Flow Increased");
    }
}

```

- `package com.design.patterns.facade.subsystem;` — placed in the `subsystem` subpackage to visually separate "subsystem internals" from the facade that fronts them.
- `open()` / `close()` — the fine-grained lifecycle operations a real client would otherwise have to call directly, in the right order, relative to the moisture sensor.
- `increaseFlowRate()` — a third independent operation; naming it clearly matters because this method name also appears in the facade's orchestration code, and a typo here would propagate everywhere it's called.
- Every method just prints what happened — this module's purpose is to show *coordination*, not real hydraulics, so the bodies stay minimal on purpose.

SoilMoistureSensor — subsystem class

```

package com.design.patterns.facade.subsystem;

public class SoilMoistureSensor {

    public void activate() {
        System.out.println("-----Soil Moisture Sensor On");
    }

    public void deactivate() {
        System.out.println("-----Soil Moisture Sensor Off");
    }
}

```

- Same shape as `ValveController` but a distinct, unrelated class — the two subsystems know nothing about each other. That independence is exactly why a coordinator (the facade) is needed: nothing else guarantees "the sensor wakes up after the valve opens, and both shut down together."

IrrigationFacade — the Facade

```

package com.design.patterns.facade;

import com.design.patterns.facade.subsystem.SoilMoistureSensor;
import com.design.patterns.facade.subsystem.ValveController;

public class IrrigationFacade {

    private ValveController valveController;
    private SoilMoistureSensor soilMoistureSensor;

    public IrrigationFacade() {
        this.valveController = new ValveController();
        this.soilMoistureSensor = new SoilMoistureSensor();
    }

    public void startWatering() {
        valveController.open();
        soilMoistureSensor.activate();
        valveController.increaseFlowRate();
    }

    public void stopWatering() {
        valveController.close();
        soilMoistureSensor.deactivate();
    }
}

```

- `package com.design.patterns.facade;` — the facade lives at the top of the `facade` package, next to the client. Nothing about "facade" needs its own subpackage: the class name already says what it is, so a `facade.facade` subpackage would only add a redundant path segment.
- `private ValveController valveController;` `private SoilMoistureSensor soilMoistureSensor;` — the facade holds references to every subsystem it coordinates. These fields are `private` : the client is never meant to reach through the facade to a subsystem instance.
- `public IrrigationFacade()` — the facade **constructs its own subsystems**. This is the detail that makes the pattern hold structurally: the client doesn't need to know `ValveController` or `SoilMoistureSensor` exist, let alone how to build them correctly. All the client needs is `new IrrigationFacade()` .
- `startWatering()` — the encoded startup sequence: valve open, then sensor on, then raise the flow rate. That ordering rule lives in exactly one place instead of being re-derived by every caller.
- `stopWatering()` — the encoded shutdown sequence: valve closed, then sensor off.
- Both methods are the **entire public surface** a client ever needs, which is the point of a facade — one intention-revealing call replaces several individually-ordered subsystem calls.

```
package com.design.patterns.facade;

public class SmartIrrigationDesignPattern {

    public static void main(String[] args) {
        System.out.println("Facade Design Pattern");

        IrrigationFacade irrigationFacade = new IrrigationFacade();
        irrigationFacade.startWatering();
        irrigationFacade.stopWatering();
    }
}
```

- No import of `ValveController` or `SoilMoistureSensor` — the client cannot even name those classes, because it never needs to construct or call them.
- `new IrrigationFacade()` — the client's only point of contact with the subsystem is through the facade's no-arg constructor.
- `irrigationFacade.startWatering(); irrigationFacade.stopWatering();` — two calls, each a full workflow. The client never issues an `open()` / `close()` / `activate()` / `deactivate()` / `increaseFlowRate()` call itself.

Why these design decisions

- **The facade constructs its own subsystems (no-arg constructor).** An earlier version of this module took `ValveController` / `SoilMoistureSensor` as constructor arguments, which forced the client to `import` and `new` both subsystem classes before it could even build the facade — that leaks exactly the knowledge a facade is supposed to hide. Making the constructor no-arg and having the facade own construction is the smallest change that closes that gap; the client now touches only `IrrigationFacade`.
- **Subsystem fields are private, not exposed via getters.** If the client could reach `irrigationFacade.getValveController().open()`, the facade would be advisory rather than structural. Keeping the fields private (with no accessors) makes the facade the only path to the subsystem.
- **Two workflow methods, not one.** `startWatering()` / `stopWatering()` mirror the natural lifecycle (you don't start an irrigation cycle without eventually ending it), and each hides a different ordering rule. A single `run()` method would hide the fact that starting and stopping are independently callable operations.
- **No facade facade subpackage.** The subsystem classes live under `facade.subsystem`, a role-named subpackage, so the `IrrigationFacade` class itself stays directly in `com.design.patterns.facade`, alongside the client — one less meaningless path segment to read past.
- **Subsystem classes are unaware of the facade.** `ValveController` and `SoilMoistureSensor` have zero references to `IrrigationFacade` or to each other. Coordination is entirely the facade's responsibility, which is what keeps the subsystem classes independently reusable (and independently testable) outside of this particular workflow.

Execution flow trace

```
SmartIrrigationDesignPattern.main
├── System.out.println("Facade Design Pattern")
├── new IrrigationFacade()
│   ├── new ValveController()      (facade builds its own subsystem)
│   └── new SoilMoistureSensor()   (facade builds its own subsystem)
├── irrigationFacade.startWatering()
│   ├── valveController.open()      → "-----Valve Controller Open"
│   ├── soilMoistureSensor.activate() → "-----Soil Moisture Sensor On"
│   └── valveController.increaseFlowRate() → "-----Valve Controller Flow Increased"
├── irrigationFacade.stopWatering()
│   ├── valveController.close()     → "-----Valve Controller Closed"
│   └── soilMoistureSensor.deactivate() → "-----Soil Moisture Sensor Off"
```

Expected output

Captured by actually running the module (`java -cp target/classes com.design.patterns.facade.SmartIrrigationDesignPattern`):

```
Facade Design Pattern
-----Valve Controller Open
-----Soil Moisture Sensor On
```

```
-----Valve Controller Flow Increased  
-----Valve Controller Closed  
-----Soil Moisture Sensor Off
```

How to run

```
cd Structural/Facade_Design_Pattern  
JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64 mvn -o clean compile  
java -cp target/classes com.design.patterns.facade.SmartIrrigationDesignPattern
```

(Drop `-o` from the Maven command only if offline dependency resolution fails.)